

STRUMPACK meets Kernel Matrices

Liza Rebrova

LBNL summer internship, Scalable Solvers Group

September, 1, 2017

Kernel matrices

Intuitively, **kernel matrices** can be viewed as **similarity matrices**

$$K(i, j) = \text{similarity score } x_i \leftrightarrow x_j,$$

where K is $n \times n$ positive semi-definite matrix, defined by a set of objects $x_1, \dots, x_n$.

The objects can be anything starting from two integers, two real valued vectors, trees whatever provided that the kernel function knows how to compare them.

We are going to deal with vectors in $\mathbb{R}^d$ only :)

Kernel examples

- **Linear kernel** Similarity is the length of the projection of one vector on another.

$$K(i, j) = \langle x_i, x_j \rangle$$

- **Gaussian kernel** Similarity is closeness in euclidean distance, h is reweighting.

$$K(i, j) = \exp \left(-\frac{\|x_i - x_j\|_2^2}{2h^2} \right)$$

- **Laplacian kernel**

$$K(i, j) = \exp \left(-\frac{\|x_i - x_j\|_2}{h} \right)$$

- **Degree d polynomial kernel**
- **Sigmoid neural network kernel**
- ...

Kernel examples

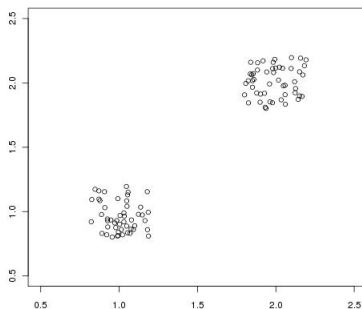
- **Gaussian kernel** Similarity is closeness in euclidean distance, h is reweighting.

$$K(i, j) = \exp \left(-\frac{\|x_i - x_j\|_2^2}{2h^2} \right)$$

Motivation

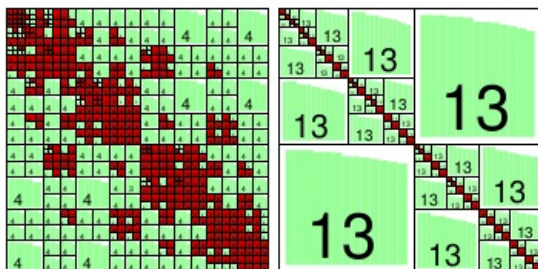
Kernel matrices are good candidates to be compressed by STRUMPACK, as

- they contain many near zero elements
- they contain many similar elements
- but they are usually full rank - ones on diagonal



HSS matrix reminder

Green blocks need to be low rank for good compression. Red blocks need to be quite small in size.



For gaussian kernel we have (i, j) small if x_i is far from x_j (green?)
 For gaussian kernel we have (i, j) large if x_i is close to x_j (red?)

(picture is taken from Alexander Litvinenko presentation slides)

Clustering

By nature of applications, we can usually assume that some points are close to each other and some are far from each other. But we cannot assume that we know where each points are.

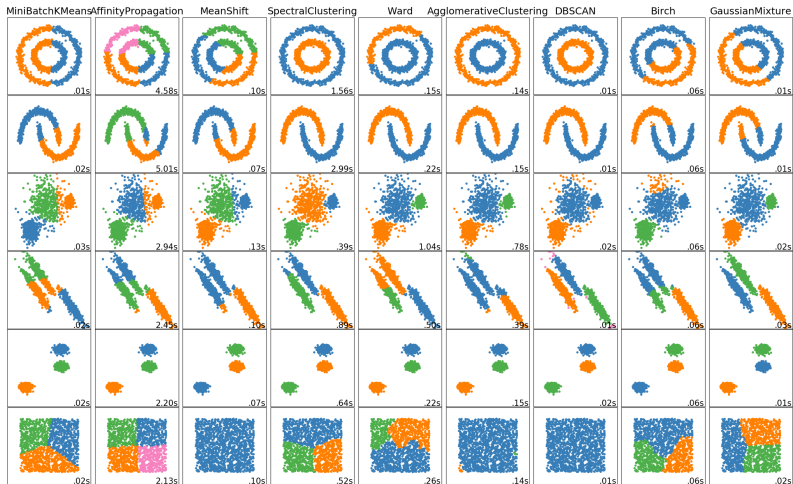
Hence, the goal is

1. find groups of points with large between-group distances and small within-group distances
2. permute matrix K so that the points of each group occupy consecutive indices $\therefore$ they will form dense diagonal blocks

This is clustering problem!

Clustering methods

Lots of them...

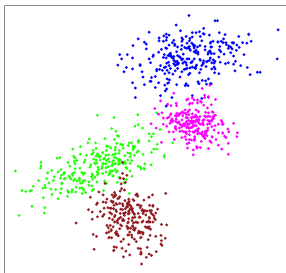


Specific requirements on clustering

Due to our final goal to make good compression of K rather than just find good separation of the points x_i , we have additional requirements:

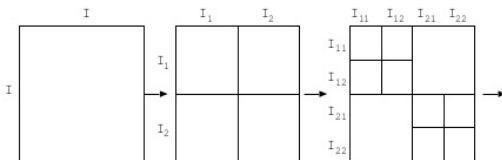
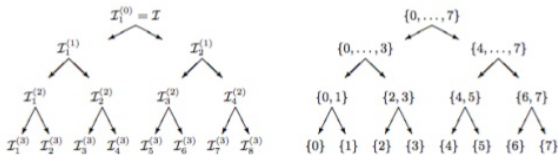
- Clusters should be small enough (otherwise dense HSS-blocks use too much memory)
- Clusters should have similar size
- We need to construct the whole hierarchical tree of embedded clusters

Example - what clusters combine first?



HSS tree reminder

Every vertex of a tree corresponds to a consecutive range of indices, representing sub-block. In every iteration this range is splitted into two children sub-ranges.



Quality measures

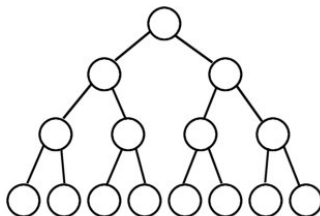
- **Memory** Sum of sizes of blocks used in compressed structure:
 U_i, V_i, B_{ij}, D_i (size = number of elements =
 $|\text{rows}| * |\text{columns}|$)
- **Maximal rank** Rank of the maximal dense block used in
HSS-structure
- **Compression quality**

$$\frac{\|A - HSS(A)\|_F}{\|A\|_F}$$

- **Prediction accuracy** will be discussed in the second part

Trees

1. Natural tree

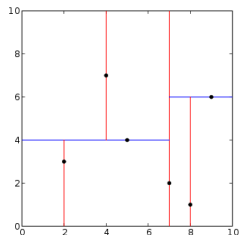


- number of levels is chosen to get good leaf size (which is heuristically ~ 16)
- does not use any information about mutual distances
- least efficient method

Divisive trees

2. KD tree

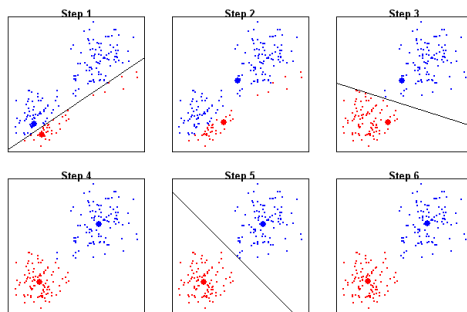
- at every step we choose some dimension (feature) $i \in \{1, \dots, d\}$, which will be the splitting pivot
- split all the points in two classes with respect to a mean/median of the column i
- works better if we choose pivot direction as a direction of maximum spread
- very simple implementation-wise, widely used in similar tasks
- requires small number of features (very tall data matrix)



Divisive trees

3. Recursive two means

a) Starts with dividing all the points into two clusters



b) Divides each of the clusters found into two using same method

c) Continues splitting until *min cluster size* achieved

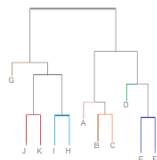
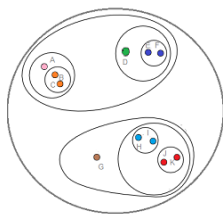
Divisive trees

3. Recursive two means - continued

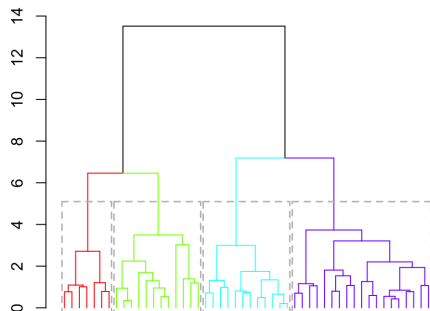
- Best in terms of memory achieved
- Quite optimal in other measures (rank, compression quality)
- Instable in rank (several runs show highly variable maximal rank, e.g. 86, 89, 94, 117, 131, 137, 151)
- Optimal *min cluster size* is ~ 16 heuristically, irregardless of data

Agglomerative trees

4. Hierarchical



Cluster Dendrogram

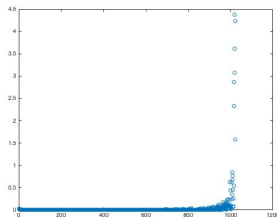


Ward linkage (weighted average distance) used:

$$\text{dist}(A, B) = \frac{|A| \cdot |B|}{|A| + |B|} \cdot \|m_A - m_B\|^2$$

Agglomerative trees

4. Hierarchical - continued



- Linkage is very important to create size-balanced clusters
- Achieves best compression quality
- Memory is a bottleneck for a hierarchical
- Hard to do in parallel (need to store distance matrix or go with rough approximations)

Agglomerative trees

5. Bottom up glueing

Other idea: start from k clusters and build down-up agglomerative tree in a balanced way

Advantages with respect to hierarchical:

- balanced tree: at every level combine clusters in pairs $\therefore$ tree will grow uniformly
- allows experiments with the methods of finding clusters (we can use something that parallels well (kmeans) or works well with large number of clusters (birch))

Disadvantage with respect to hierarchical:

- linkage tree is greedy, not optimal

Quality measures

- **Memory** Sum of sizes of blocks used in compressed structure:
 U_i, V_i, B_{ij}, D_i (size = number of elements =
 $|\text{rows}| * |\text{columns}|$)
- **Maximal rank** Rank of the maximal dense block used in
HSS-structure
- **Compression quality**

$$\frac{\|A - HSS(A)\|_F}{\|A\|_F}$$

- **Prediction accuracy** will be discussed in the second part

Datasets and comparison

All the natural datasets used are taken from UCI repository

- SUSY (4500, 8)
- PENDIGITS (3495, 16)
- WINEQUALITY (4898,12)
- LETTER (5000, 16)
- ABALONE (4177, 8)
- ABALONE normalized (4177, 8)
- COVTYPE (4500, 54)

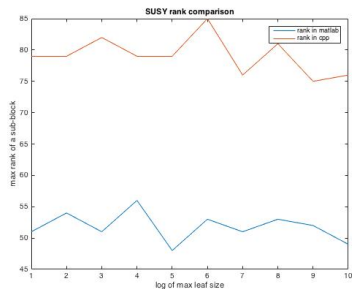
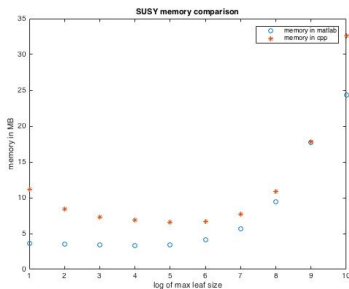
Synthetic datasets (mixture of several 8-dim gaussians) were used to explore influence of the presence of cluster structure to the various compression methods

Some observations and heuristic conclusions

- **Recursive two means** (divisive method) is best in sense of memory usage and seems most optimal
- In case of strong *cluster structure* in data **agglomerative methods** (hierarchical and down-up glueing) give very low rank, due to finding real clusters
- **Hierarchical clustering** looks to be best in terms of compression quality (by one order)
- When one coordinate is considerable more spread than the others, **kd-tree** works really well
- **Natural tree** is the simplest to implement :)

Further directions

- Optimize performance of recursive two means (divisive) in terms of number of iterations needed
- Choose the best place to stop growing the tree (now it seems that irregardless of data we should stop at leaf size ~ 20)



Further directions

continued...

- Improve stability of 2means performance (??)
- Find "the best" agglomerative tree and use it for the data with assumed cluster structure (like handwritten digits, letters, etc)
- Understand why agglomerative clusterings does not achieve memory as small as divisive 2 means: is this an inefficiency of implementation (that is generally harder) or an HSS algorithm feature? (e.g. sparse upper level blocks are more important)
- Suggestions?

Kernel Ridge Regression

= Ridge Regression + Kernel Trick

1. Kernel trick

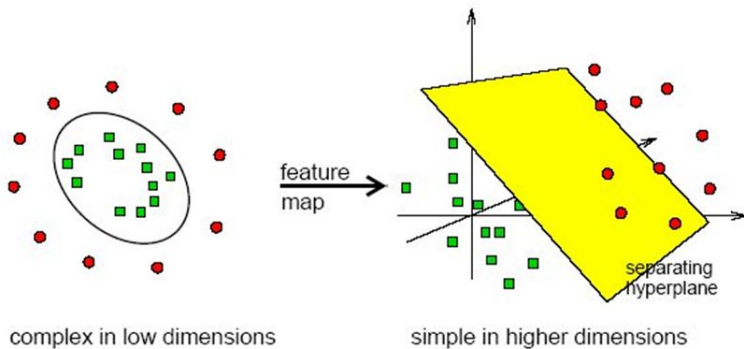
- map points to higher-dimensional feature space using function $\varphi(x)$ (e.g. $x \mapsto \varphi(x) = [x, x^2, x^3]$).
- replace scalar product in the new space by kernel function K

$$K(x_i, x_j) = \langle \varphi(x_i), \varphi(x_j) \rangle.$$

Properties of K :

- always positive semi-definite
- often much faster to compute than $\varphi(x)$ themselves
- can be applied if method depends on scalar products

Separation can be easier in higher dimensions



(picture is found in Radha Chitta presentation on Kernel K-Means)

Kernel matrix can be easy to compute

Suppose higher-dimensional features are second order terms of x'_i s:

$$\varphi(x) = \begin{bmatrix} x_1 x_1 \\ x_1 x_2 \\ x_2 x_1 \\ \dots \\ x_n x_n \end{bmatrix}$$

Then the scalar product

$$\begin{aligned} K(x, z) &:= \varphi(x)^T \varphi(z) = \sum_{i,j=1}^n (x_i x_j)(z_i z_j) = \\ &= \left(\sum_{i=1}^n x_i z_i \right) \left(\sum_{j=1}^n x_j z_j \right) = (x^T z)^2. \end{aligned}$$

Note: calculating the high-dimensional $\varphi(x)$ requires $O(n^2)$ time, finding $K(x, z)$ takes only $O(n)$ time - linear in the dimension of the input attributes.

Kernel Ridge Regression

2. Ridge regression

We need to minimize the cost function:

$$C(\mathbf{w}) = \sum_i (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|^2 \rightarrow \min$$

Differentiating

$$C'(\mathbf{w})_j = 2 \sum_i (y_i - \mathbf{w}^T \mathbf{x}_i) \mathbf{x}_{ij} + 2\lambda \mathbf{w}_j = 0$$

In matrix form we get that the argmin is

$$\mathbf{w} = (\lambda I + X^T X)^{-1} X^T \mathbf{y},$$

where X - train matrix, y - a vector of train labels.

So, we do regression with the optimal **weights**

$$\mathbf{w} = X^T(\lambda I + X^T X)^{-1} \mathbf{y}$$

Prediction step: given x' - a vector from the test set,

$$\begin{aligned} y' &:= \mathbf{w}^T \mathbf{x}' = \\ &= [(\lambda I + X^T X)^{-1} \mathbf{y}]^T X \mathbf{x}' = \\ &= [(\lambda I + \langle X, X \rangle)^{-1} \mathbf{y}]^T \cdot \langle X, \mathbf{x}' \rangle \end{aligned}$$

Note: regularization term λ helps to stabilize the inverse numerically by bounding the smallest eigenvalues away from zero.

So, we do regression with the optimal **weights**

$$\mathbf{w} = X^T(\lambda I + X^T X)^{-1} \mathbf{y}$$

Prediction step: given x' - a vector from the test set,

$$\begin{aligned} y' &:= \mathbf{w}^T \mathbf{x}' = \\ &= [(\lambda I + X^T X)^{-1} \mathbf{y}]^T X \mathbf{x}' = \\ &= [(\lambda I + \mathcal{K}(X, X))^{-1} \mathbf{y}]^T \cdot \mathcal{K}(X, \mathbf{x}') \end{aligned}$$

Note: regularization term λ helps to stabilize the inverse numerically by bounding the smallest eigenvalues away from zero.

Algorithm for binary classification with the gaussian kernel

1. Compute kernel matrix K :

$$K(i, j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|_2^2}{2h^2}\right)$$

for all $\mathbf{x}_i, \mathbf{x}_j$ from the train set

2. Compute the weights by solving linear system ($\mathbf{y}$ - train labels)

$$\mathbf{v} : \quad \mathbf{y} = (\lambda I + K)\mathbf{v}$$

3. Compute kernel vector K' for the test vector $\mathbf{x}'$:

$$K'(i) = \exp(-\|\mathbf{x}_i - \mathbf{x}'\|_2^2/2h^2)$$

4. Predict the sign

$$y' := \text{sign}\langle \mathbf{v}, K' \rangle$$

Scaling, paralleling, speeding kernel computations

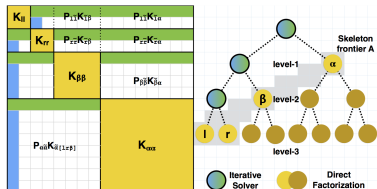
...is hard.

Some literature:

1. An NlogN Parallel Fast Direct Solver for Kernel Matrices (Yu, March, Biros, 2017)
2. Structured Block Basis Factorization for Scalable Kernel Matrix Evaluation (Wang, Li, Mahoney, Darve, 2015)
3. Learning the kernel matrix via predictive low-rank approximations (Straar, Curk, 2016)
4. and more ...

$$M = \tilde{U} \tilde{C} \tilde{V}^T$$

Biros et al paper



Dataset	N	d	h	λ	Acc
COVTYPE	0.1–0.5M	54	.07	.3	96%
SUSY	4.5M	8	.07	10	78%
MNIST2M	1.6M	784	.30	0	100%
HIGGS	10.5M	28	.90	.01	73%
MRI	3.2M	128	3.5	10	-
MNIST8M	8.1M	784	1.0	1.0	-
NORMAL	1–32M	64	.19	1.0	-

SUSY dataset:

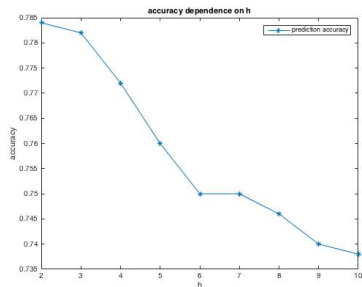
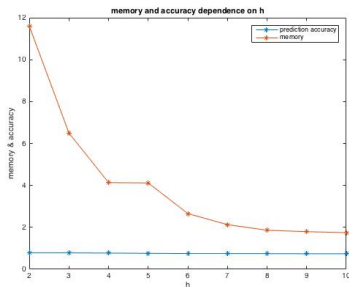
features - kinematic properties measured by the particle detectors in the accelerator, binary targets - 1 for signal, 0 for background)

$$\text{Acc} = \text{prediction accuracy} = \frac{\text{number of misclassified}}{\text{test data size}} \sim 0.78$$

(pictures are from Yu, March, Biros's paper)

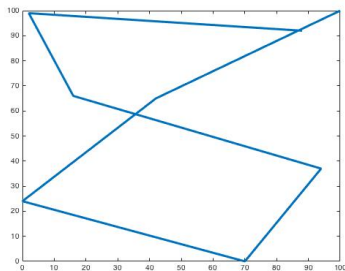
SUSY and STRUMPACK

- We took a subset of the data:
4500 points in the train set, 500 points in the test set (Biros et al: $4.5e+6$ points - train set, 10000 - test set)
- Used $\lambda = 10$, tolerance = $1e - 2$
- Accuracy depends on parameter h and solution quality $\lesssim 1e - 2$ (does not improve for better quality)
- We obtained 0.79 accuracy for $h = 3$, memory = 3.05MB, max rank = 65.



PENDIGITS example

- 7494 points in the train set
- 3498 points in the test set
- every data point $\{x_1, y_1, x_2, y_2, \dots, x_8, y_8\} \in \mathbb{R}^{16}$ contains 8 coordinates (x_i, y_i) regularly spaced in arc length along a written digit



PENDIGITS challenge: multi-class prediction

1. We had 10 classifiers of the type "two or not two".
 - Need to assign "most likely class". So, in step 4

$$\text{sign}\langle \mathbf{v}, K' \rangle \mapsto \text{argmax}_{t=1\dots 10} \langle \mathbf{v}_t, K' \rangle$$

gives the most confidently predicted number.

2. Another way: create $\frac{10 \cdot 9}{2}$ classifiers "rather two or three".
 - Get 45 predicted numbers
 - Final prediction: by the majority vote
 - It should work, as there are at most 9 times we can predict each number, and we believe that real number will be predicted all 9 possible times (hence, others less)
 - Requires 45 smaller training sets (and kernel matrices) $\therefore$ more memory than one original training set (and one kernel matrix)

PENDIGITS - parameters and results

Parameters:

- tolerance = 1e-03
- regression parameters: $\lambda = 10$, $h = 5$
- number of gaussian samples = 500
- kernel = gaussian
- tree type = top down recursive 2-means tree

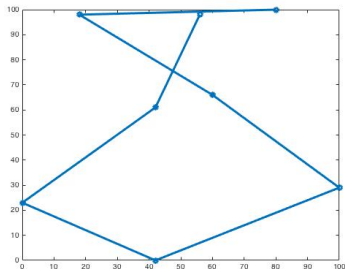
Results:

- Memory used : 4.630 MB
- Memory 1.031 percent of dense
- Max rank in compression: 52
- Compression quality: 5e-06
- Accuracy = 0.9774 (79 points were misclassified out of 3498)

Further questions

- Finding optimal λ, h - cross validation - ? (requires repeating algorithm many times)
- Comparison with the existing memory/time on large datasets where compression is crucial
- Look at some other kernel methods and kernel functions
- Suggestions?

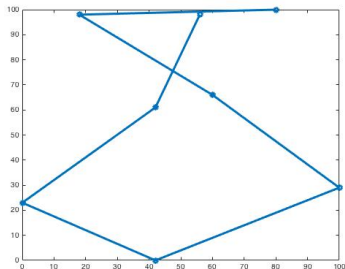
PENDIGITS - guess the number:)



?

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

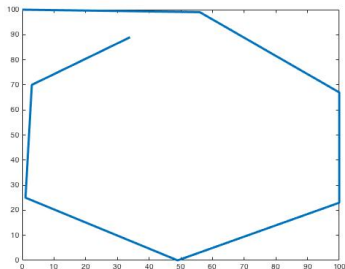
PENDIGITS - guess the number:)



8

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

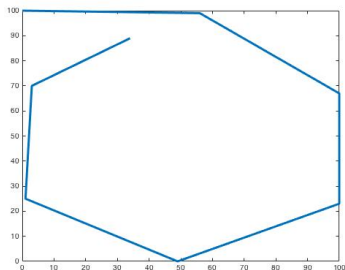
PENDIGITS - guess the number:)



?

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

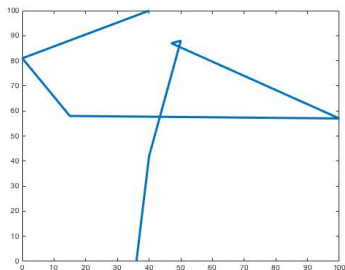
PENDIGITS - guess the number:)



0

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

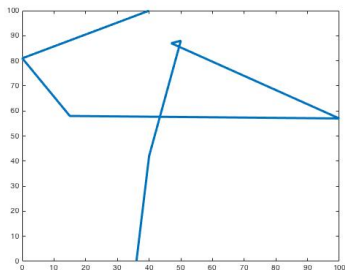
PENDIGITS - guess the number:)



?

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

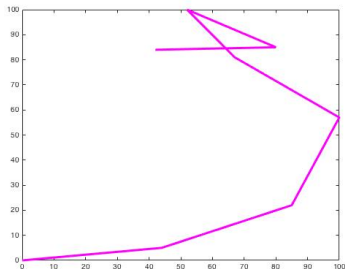
PENDIGITS - guess the number:)



4

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

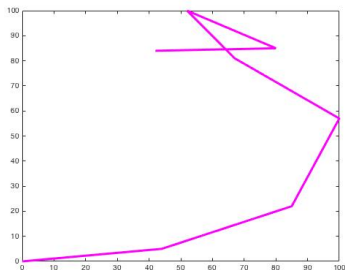
PENDIGITS - guess the number:)



?

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

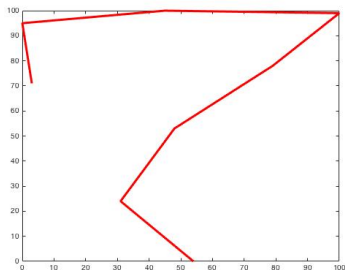
PENDIGITS - guess the number:)



9

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

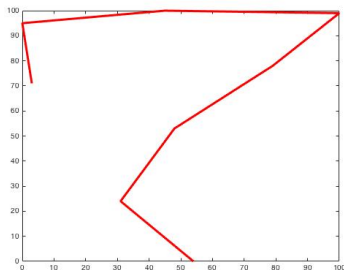
PENDIGITS - guess the number:)



?

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

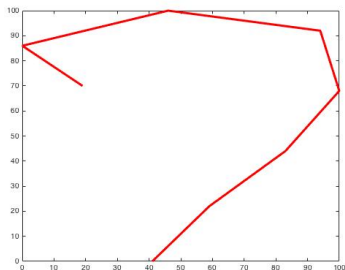
PENDIGITS - guess the number:)



7 (predicted 3)

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

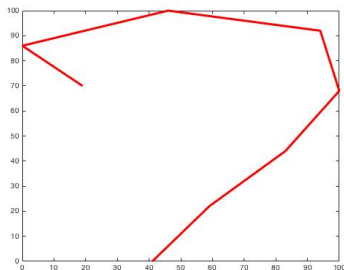
PENDIGITS - guess the number:)



?

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

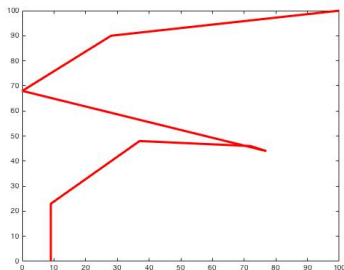
PENDIGITS - guess the number:)



9 (predicted 7)

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

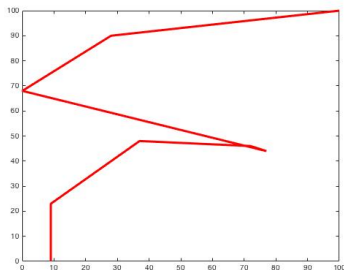
PENDIGITS - guess the number:)



?

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

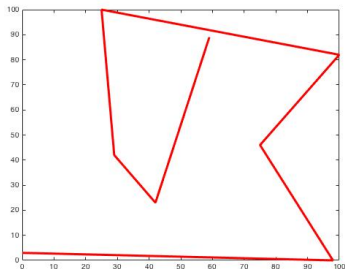
PENDIGITS - guess the number:)



4 (predicted 5)

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

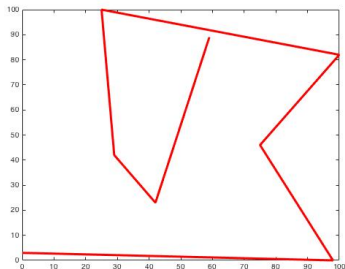
PENDIGITS - guess the number:)



?

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

PENDIGITS - guess the number:)



3 (predicted 9)

Blue numbers - predicted correctly, magenta - correct for some experiments, red - errors

Thank you all for the great summer! :)